

W01 — Manual da Demo (Produção)

Enterprise RAG: DataOps Knowledge Hub | Walkthrough em Produção

Pré-Requisitos

Antes de iniciar a demo de produção:

- ☐ Deploy executado com sucesso (prompt 14 completo)
 - ☐ URL pública disponível: `https://<production-url>`
 - ☐ `/health` retornando healthy
 - ☐ Ingestion executada em produção
 - ☐ MCP configurado no Claude Code apontando para URL de produção
 - ☐ Data generator rodando (dados frescos)
-

Contexto: Transição Dev → Prod

Após a demo local, o instrutor faz a transição:

“Tudo funciona local. Mas local é o meu laptop. Se eu fechar, acabou. Agora vamos colocar isso em produção — disponível ²⁴/7, com URL pública, acessível por qualquer agente no planeta.”

Parte 1 — Mostrar o Deploy

1.1 Executar o Deploy (5 min)

```
# Se Railway:
railway up

# Se VPS/Docker:
bash scripts/deploy.sh
```

Mostrar o output do deploy — services subindo, health checks passando.

“Um comando. Tudo em produção.”

1.2 Mostrar a URL Pública (2 min)

```
# Health check em produção
curl -s https://<production-url>/health | python3 -m json.tool
```

Mostrar: todos os services healthy, agora com URL pública.

“Esse endpoint está acessível de qualquer lugar do mundo agora.”

1.3 Swagger UI em Produção (1 min)

Abrir no browser: `https://<production-url>/docs`

“A mesma documentação, agora pública. Qualquer dev da empresa pode ver e consumir.”

Parte 2 — Queries em Produção

2.1 Ledger — Text-to-SQL (2 min)

```
curl -s -X POST https://<production-url>/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Quais são os top 5 clientes do plano enterprise por receita total?", "sources": ["ledger"]}' \
  | python3 -m json.tool
```

“Mesma query em português, agora batendo no Postgres em produção. Dados reais, crescendo continuamente.”

2.2 Memory — Vector Search (2 min)

```
curl -s -X POST https://<production-url>/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Qual é a política de retenção de dados PII?", "sources": ["memory"]}' \
  | python3 -m json.tool
```

2.3 Brain — Graph Traversal (2 min)

```
curl -s -X POST https://<production-url>/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Quem é o owner do pipeline etl_billing_daily e quais tabelas ele lê?", "sources": ["brain"]}' \
  | python3 -m json.tool
```

2.4 Cross-Domain (2 min)

```
curl -s -X POST https://<production-url>/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Quais são os top clientes por faturamento, qual o SLA do pipeline de billing, e qual seria o impacto se a tabela orders ficasse indisponível?"}' \
  | python3 -m json.tool
```

“Produção. 3 data stores. Roteamento inteligente. Perguntas em português. JSON validado. Tudo funcionando.”

Parte 3 — Grand Finale: Claude Code + MCP em Produção

3.1 Mostrar a Config do MCP (2 min)

Abrir o `mcp-config.json` e mostrar:

```
{
  "mcpServers": {
    "dataops-knowledge-hub": {
      "command": "python",
      "args": ["-m", "src.mcp.run"],
      "env": {
        "API_BASE_URL": "https://<production-url>"
      }
    }
  }
}
```

“A única diferença entre dev e prod é essa URL. O MCP aponta para produção agora. Qualquer Claude Code no mundo pode consumir esse sistema.”

3.2 Abrir Nova Sessão do Claude Code (1 min)

```
claude
```

“Sessão nova. Limpa. Ele não sabe nada sobre o projeto. Mas ele tem uma tool.”

3.3 Perguntas Progressivas (10 min)

Pergunta 1 — Health Check:

```
Verifica o status da plataforma DataOps. Está tudo saudável?
```

Claude chama `check_platform_health` → mostra todos os services healthy em produção.

Pergunta 2 — Ledger (simples):

```
Quantos clientes enterprise temos e qual o faturamento total?
```

Claude chama `query_knowledge_hub` → Ledger responde com SQL.

Pergunta 3 — Memory:

```
Qual a política de retenção de dados PII? Quanto tempo podemos manter dados pessoais?
```

Claude chama → Memory responde com citação do documento.

Pergunta 4 — Brain:

```
Quem é o owner do pipeline etl_billing_daily e quais tabelas ele lê?
```

Claude chama → Brain traversa o grafo → mostra dependências.

Pergunta 5 — CROSS-DOMAIN (O Grand Finale Final):

Preciso de um relatório executivo completo:

1. Quais são os top 5 clientes por faturamento
2. Qual o SLA do pipeline que processa os dados deles
3. Qual seria o impacto operacional se a tabela de orders ficasse indisponível
4. Quais ações recomendadas para mitigar o risco

Formate como um relatório profissional.

Esperar. Claude Code chama a tool → Router decompõe → 3 engines → resposta sintetizada → Claude formata como relatório.

3.4 O Momento Meta (2 min)

“Parem e pensem no que acabou de acontecer:

1. Nós usamos o Claude Code para **CONSTRUIR** esse sistema (prompts 1-14)
2. O sistema está em **PRODUÇÃO** (URL pública, dados reais)
3. Agora o Claude Code **CONSOME** o sistema que ele mesmo construiu

Isso é o loop completo. O AI-Native Engineer orquestra agentes que constroem sistemas que são consumidos por agentes. Vocês acabaram de ver isso acontecer ao vivo.”

Parte 4 — Encerramento e Gancho para W02

4.1 Recap (3 min)

“O que vocês construíram hoje:

- Um RAG enterprise com 3 paradigmas de retrieval (SQL, Vector, Graph)
- Um pipeline de dados real com geração contínua
- Uma API production-grade com Pydantic contracts
- Um MCP Server que qualquer agente pode consumir
- Tudo em produção, rodando ²⁴/₇

Isso não é toy example. Isso é o que um AI-Native Engineer entrega.”

4.2 Gancho para W02 (2 min)

“Mas tem um problema: esse sistema é PASSIVO. Ele só responde quando alguém pergunta. E se ele pudesse MONITORAR ativamente? Detectar anomalias? Alertar quando um SLA está em risco?

No próximo workshop, o Rafael vai pegar essa API e construir um AGENTE AUTÔNOMO com LangGraph que monitora a qualidade dos dados ²⁴/₇. O que vocês construíram hoje é a fundação de tudo que vem depois.”

Timing Total da Demo de Produção

Parte	Duração	Acumulado
1. Deploy + URL pública	8 min	8 min
2. Queries em produção	8 min	16 min
3. Grand Finale (Claude Code + MCP)	15 min	31 min
4. Encerramento + Gancho W02	5 min	36 min

Total: ~36 minutos

Contingências

Problema	Solução
Deploy falha	Usar a stack local como fallback. “Em produção seria igual, mas vamos mostrar local.”
URL demora para responder	“Cold start. Primeira request demora. Vejam a segunda...”
OpenAI rate limit em prod	Trocar para <code>gpt-4.1-nano</code> no <code>.env</code> de produção
MCP não conecta com prod	Mudar <code>API_BASE_URL</code> para localhost. Funciona igual.
Tempo estourando	Pular queries individuais (Parte 2), ir direto pro Grand Finale (Parte 3)

Diferenças entre Demo Dev e Demo Prod

Aspecto	Dev	Prod
URL	<code>http://localhost:8000</code>	<code>https://<production-url></code>
Neo4j UI	<code>http://localhost:7474</code> (mostra grafo)	Não exposto (segurança)
Qdrant Dashboard	<code>http://localhost:6333/dashboard</code>	Não exposto (segurança)
Dados	Gerados desde o <code>make up</code>	Gerados desde o deploy (mais frescos)
MCP config	<code>API_BASE_URL=http://localhost:8000</code>	<code>API_BASE_URL=https://<production-url></code>
Propósito	Mostrar as peças por dentro	Mostrar que funciona no mundo real